```
        "appsettings.json",
        optional: false)
    .AddJsonFile(
        $"appsettings.{env}.json",
        Optional: true);

if(env.IsDevelopment())                                    ❶
{
    builder.Configuration.AddUserSecrets<Program>();       ❷
}

WebApplication app = builder.Build();

app.MapGet("/", () =>"Hello world!");

app.Run();
```

❶ Extension methods make checking the environment simple and explicit.

❷ In Staging and Production, the User Secrets provider won't be used.

As already mentioned, it's also common to customize your application's middleware pipeline based on the environment. In chapter 4 you learned that `WebApplication` adds the `DeveloperExceptionPageMiddleware` conditionally when developing locally. The following listing shows how you can use `IHostEnvironment` to control your pipeline in this way so that when you're in staging or production, your app uses `ExceptionHandlerMiddleware` instead.

Listing 10.15 Using the hosting environment to customize your middleware pipeline

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.AddProblemDetails();                ❶

WebApplication app = builder.Build();

if (!builder.Environment.IsDevelopment())   ❷
{                                           ❷
    app.UseExceptionHandler();              ❷
}                                           ❷

app.MapGet("/", () =>"Hello world!");
```